Manufacturing Operational Insights Report *Dockerized Sports Analytics Application*

Course Name: Data Analytics

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	ABHAY SINGH THAKUR	EN22CS304003
2	SARTHAK GOKHALE	EN22CS304057
3	HIMANSHI PORWAL	EN22CS306021
4	AKASH MUVEL	EN23EE3L1001
5	PRATHAM JOSHI	EN22CS306037

Group Name: ***Group 04D11***

*Project Number:****DO-30***

Industry Mentor Name:

University Mentor Name:

*Academic Year:****2022-2026***

Problem Statement & Objectives

1. Problem Statement : **Dockerized Sports Rust Service**

2. Project Objectives:

   Containerize a Sports Service (built with Rust) using Docker best practices. create **multi-stage Dockerfiles** to make the image **smaller, faster, and more secure**. We also use **Docker Compose** to run the Sports service along with its **database** in one setup. This helps the team get the **same environment** on every system, and makes the application **portable** for both development and production.

3. Scope of the Project
   - Containerize the Sports Service application using Docker.
   - Create optimized multi-stage Dockerfiles.
   - Configure Docker Compose for backend and MongoDB.
   - Ensure consistent development and production environments.
   - Improve portability and team collaboration.
   - Prepare the application for future cloud deployment and CI/CD integration.

Proposed Solution

1. Key features
   - *REST API with health endpoint*
   - *Static and database-driven sports data*
   - *MongoDB integration*
   - *Docker multi-container setup*
   - *Automatic database seeding*
   - *Environment variable configuration*

2. Overall Architecture / Workflow
   - The user accesses the frontend (React + Vite).
   - Frontend sends API requests to backend services.
   - Backend processes requests and connects to MongoDB.
   - MongoDB returns data to the backend.
   - Backend sends response back to frontend.
   - Docker Compose manages all services (Frontend, Backend, MongoDB).

3. Tools & Technologies Used (*If applicable)*

- *Node.js & Express*
- *MongoDB*
- *React (Vite)*
- *Docker*
- *Docker Compose*
- *Git & GitHub*

- *AWS (for future deployment)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*

   Docker containers running successfully (Frontend, Backend, MongoDB):

```
>>
CONTAINER ID   IMAGE                          COMMAND              CREATED        STATUS              PORTS                                          NAMES
d9d57664aef9   himanshi1409/sports-frontend:latest  "docker-entrypoint.s…"  43 minutes ago  Up 43 minutes       0.0.0.0:3000->4173/tcp, [::]:3000->4173/tcp    sports-frontend
7197aca9e375   sports-rust-service-backend    "docker-entrypoint.s…"  43 minutes ago  Up 43 minutes       0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    sports-backend
f0141306e216   mongo:6                        "docker-entrypoint.s…"  43 minutes ago  Up 43 minutes (healthy)  0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp  sports-mongo
PS C:\Users\himan\Desktop\G04\sports-rust-service>
```

   Check Backend Health API

```
StatusCode        : 200
StatusDescription : OK
Content           : {"status":"OK","timestamp":"2026-02-15T15:55:34.806Z"}
RawContent        : HTTP/1.1 200 OK
                    Access-Control-Allow-Origin: *
                    Connection: keep-alive
                    Keep-Alive: timeout=5
                    Content-Length: 54
                    Content-Type: application/json; charset=utf-8
                    Date: Sun, 15 Feb 2026 15:55:34 GMT
```

   Check Static Sports Data

```
StatusCode        : 200
StatusDescription : OK
Content           : [{"name":"Cricket","players":11},{"name":"Football","players":11},{"name":"
Basketball","players":5},{"name":"Tennis","players":1},{"name":"Volleyball","players":6}]
RawContent        : HTTP/1.1 200 OK
                    Access-Control-Allow-Origin: *
```

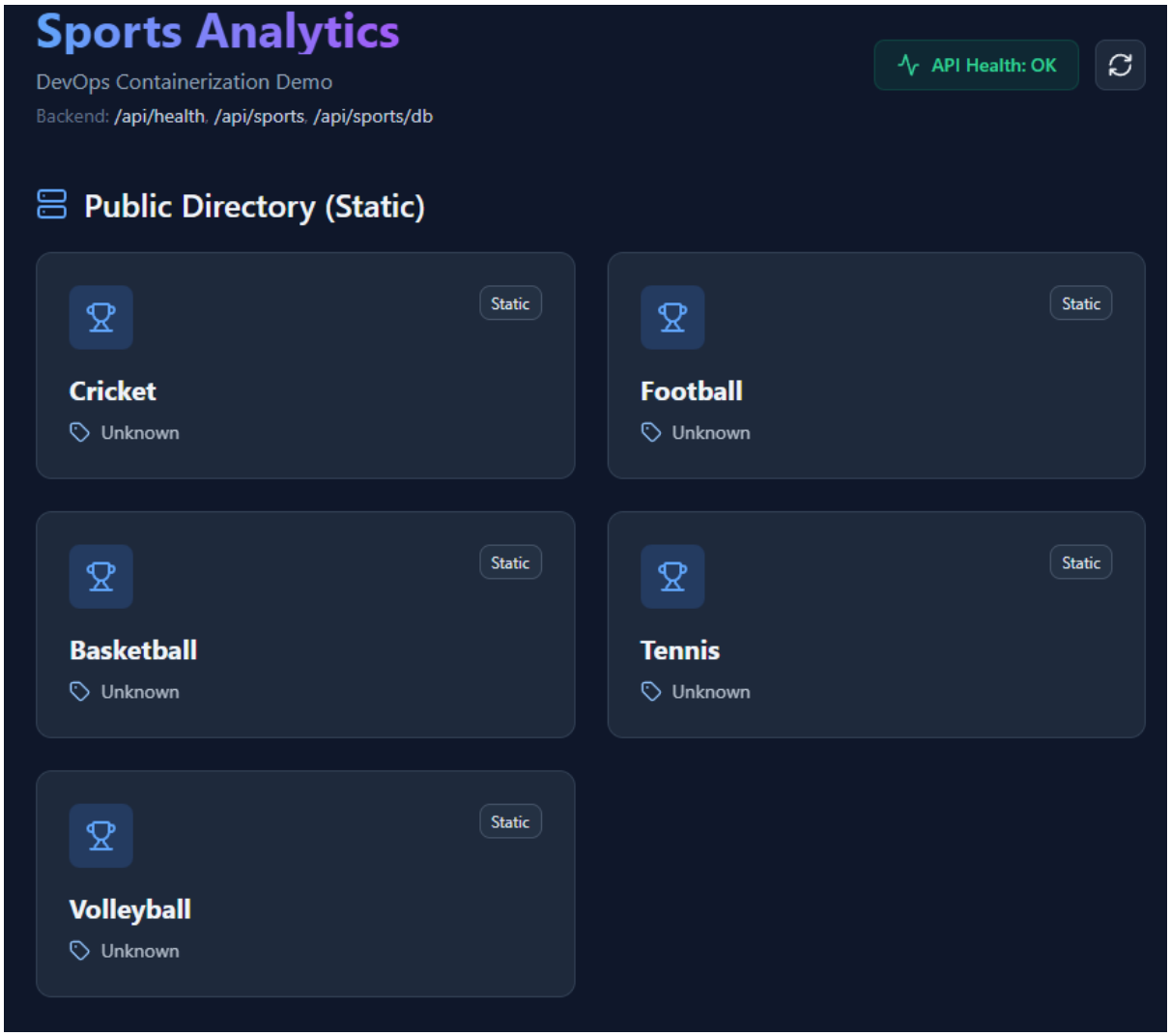
   Check Database Data

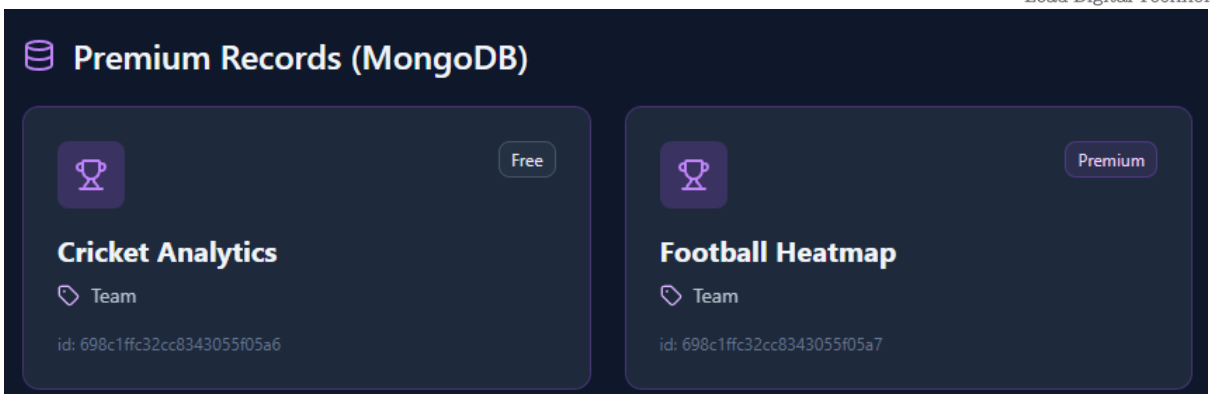
```
StatusCode        : 200
StatusDescription : OK
Content           : [{"_id":"698c1ffc32cc8343055f05a6","name":"Cricket Analytics","category":"
                    Team","premium":false,"__v":0},{"_id":"698c1ffc32cc8343055f05a7","name":"F
                    ootball Heatmap","category":"Team","premium":true,"_...
RawContent        : HTTP/1.1 200 OK
                    Access-Control-Allow-Origin: *
                    Connection: keep-alive
```

   Push Images to Docker Hub

```
IMAGE                                  ID            DISK USAGE   CONTENT SIZE   EXTRA
himanshi1409/sports-backend:latest     9686760d146b       211MB         50.7MB
himanshi1409/sports-frontend:latest    55577d282dc9      1.62GB          393MB     U
mongo:6                                03cda579c8ca      1.06GB          273MB     U
sports-rust-service-backend:latest     a4220f3fdaff       211MB         50.7MB     U
sports-rust-service-frontend:latest    3e7547fb6ae3      1.62GB          393MB
```

2. *Reports / dashboards / models*





3. *Key outcomes*

- *Full-stack application successfully containerized*
- *Multi-stage Docker build optimized image size*
- *MongoDB integrated via Docker Compose*
- *Portable & consistent environment*
- *Deployment-ready for AWS / Kubernetes*

Conclusion

In this project, we successfully containerized the Sports Analytics application using Docker and Docker Compose. The backend, frontend, and MongoDB were connected and ran smoothly inside containers.

We learned how to build Docker images, manage multiple services, fix networking issues, and make the application portable. This project helped us understand real-world DevOps practices and containerization concepts in a simple and practical way.

Future Scope & Enhancements

- Deploy the application on **AWS / Cloud platform** for real-world hosting.
- Implement **Kubernetes** for container orchestration and scaling.
- Add **CI/CD pipeline** using GitHub Actions for automatic build and deployment.
- Improve security by adding **authentication and role-based access control**.